

AI-Assisted Text Classification API

A backend service that automatically classifies incoming text into Complaint, Query, Feedback, or Other — built with Node.js, Express, and Groq's AI model, with a resilient fallback layer for guaranteed uptime.

Prepared By

Susmita Das

Submitted To

Indus Net Technologies
(INT) — Tech Hiring
Team

Track

.NET / Java / Node.js /
PHP (Backend)

Date

September 2026

01

Project Overview & Objective

This project implements a REST API that reads a piece of free-form text and automatically classifies it into one of four business-relevant categories — **Complaint**, **Query**, **Feedback**, or **Other** — along with a confidence score. The goal was to design a service that is not only functionally correct, but also production-minded: validated inputs, predictable error responses, and graceful degradation if the AI provider is slow or unavailable.

The assignment specifically called for an AI-assisted approach, so the core classification logic is delegated to a large language model (Groq's `openai/gpt-oss-20b`), while the surrounding application layer is engineered by hand to keep the system reliable, testable, and easy to extend.

Goals • Accurate 4-way text classification via AI • Strict input validation and clear error contracts • Never fail the caller — fallback logic on AI failure • Clean, modular, industry-standard code structure • Fully documented and testable via Postman	Tech Stack • Runtime: Node.js • Framework: Express.js • AI Provider: Groq API (openai/gpt-oss-20b) • HTTP Client: Axios • Testing: Postman Collection (7 scenarios)
--	---

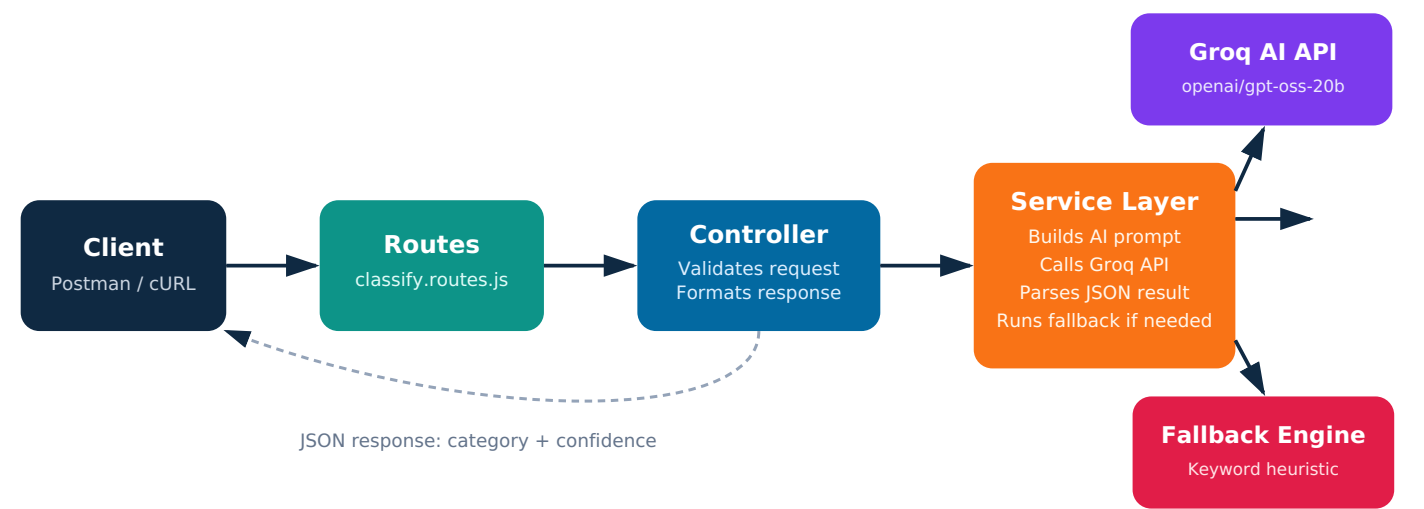
Project Folder Structure

```
AI-Assisted-Text-Classification/
├─ src/
│  ├─ controllers/
│  │   └─ classify.controller.js  # Request validation + response handling
│  ├─ services/
│  │   └─ classify.service.js    # AI call + fallback classification logic
│  └─ routes/
│     └─ classify.routes.js      # Route definitions
├─ app.js                      # Express app setup (middleware, routes)
├─ server.js                   # Entry point
├─ .env.example
├─ .gitignore
├─ package.json
├─ postman_collection.json
└─ README.md
```

02

System Architecture

The application follows a clean, layered Express.js architecture. Each layer has a single responsibility, which keeps the codebase easy to test, debug, and extend with new categories or providers in the future.



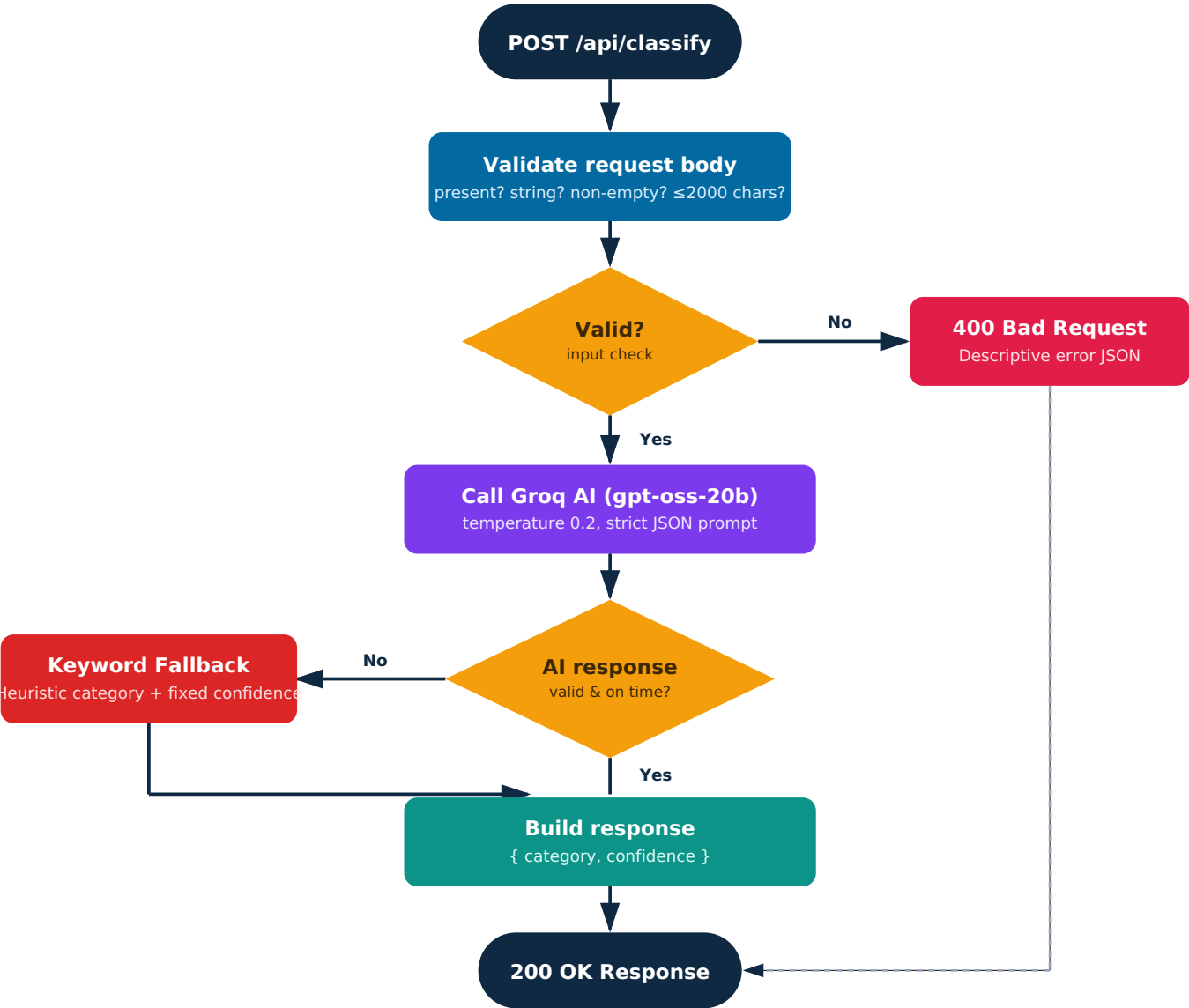
Layer Responsibilities

Layer	File	Responsibility
Entry Point	server.js	Starts the HTTP server and loads environment configuration.
App Setup	app.js	Registers middleware (JSON body parsing) and mounts the classify route.
Routing	classify.routes.js	Maps POST /api/classify to the controller.
Controller	classify.controller.js	Validates the request body (missing field, wrong type, empty string, length limit) and shapes the HTTP response.
Service	classify.service.js	Calls the Groq model with a structured prompt, parses the JSON reply, and falls back to a keyword heuristic if the AI call fails or returns malformed data.

03

Request Lifecycle — Development & Execution Flow

The diagram below traces exactly what happens from the moment a request hits the API to the moment a response is returned, including the resilience path that guarantees the caller always gets a usable answer.



04

API Contract & How AI Was Used

Endpoint

POST

/api/classify

Sample Request

```
{  "text": "My order hasn't arrived in 2 weeks, this is unacceptable."}
```

Sample Response

```
{  "category": "Complaint",  "confidence": 0.95}
```

Error Contract

Status	Scenario	Response
400	Missing <code>text</code> field	<code>{"error": "Field \"text\" is required."}</code>
400	<code>text</code> is not a string	<code>{"error": "Field \"text\" must be a string."}</code>
400	<code>text</code> is empty	<code>{"error": "Field \"text\" cannot be empty."}</code>
400	Exceeds 2000 characters	<code>{"error": "...exceeds maximum length..."}</code>
503	AI provider unavailable / timeout	<code>{"error": "AI classification service is temporarily unavailable..."}</code>
500	Unexpected server error	<code>{"error": "Failed to classify text..."}</code>

How AI Was Used

The input text is sent to Groq's `openai/gpt-oss-20b` model with a system prompt instructing it to classify the text into exactly one of four labels and return a strict JSON object containing `category` and `confidence` . A low **temperature (0.2)** was deliberately chosen to keep classifications consistent and repeatable for similar inputs, rather than creative or varied — which matters for a business-facing classification task.

To make the service production-safe, a **fallback classification engine** (a keyword-based heuristic in `classify.service.js`) activates automatically whenever the AI response is missing, malformed, unparseable, or the request itself times out or fails. This guarantees the API never leaves the caller without a category and a confidence score — even during an AI provider outage.

05

Environment Setup & How to Run the Project

The project is designed to be up and running on a fresh machine in under five minutes, with no hidden configuration steps. All secrets are kept out of source control via a `.env` file, and an `.env.example` template is committed so a reviewer knows exactly which variables to set.

Step-by-step setup

Step	Command / Action	Purpose
1. Clone the repository	<code>git clone <repo-url> && cd AI-Assisted-Text-Classification</code>	Pulls the source code locally
2. Install dependencies	<code>npm install</code>	Installs Express, Axios, and dev dependencies
3. Configure environment	<code>cp .env.example .env</code>	Creates a local, git-ignored env file
4. Add credentials	Edit <code>.env</code> : set <code>PORT</code> and <code>AI_API_KEY</code>	Supplies the Groq API key the service needs
5. Run the server	<code>npm run dev</code>	Starts the API on <code>http://localhost:5000</code>
6. Verify it's live	Console prints <code>Server running on http://localhost:5000</code>	Confirms successful startup

Environment Variables

- `PORT` — port the server runs on (default: 5000)
- `AI_API_KEY` — Groq API key, free at console.groq.com/keys

Try it Instantly

```
curl -X POST \
  http://localhost:5000/api/classify \
  -H "Content-Type: application/json" \
  -d '{"text":"What time do
you open tomorrow?"}'
```

A ready-made Postman collection (`postman_collection.json`) is also included in the repository root — import it directly into Postman to run all seven test scenarios without typing a single request by hand.

06

Testing & Verification

A complete Postman collection (`postman_collection.json`) accompanies the codebase, covering seven scenarios: one success case per category (Complaint, Query, Feedback, Other) and three deliberate failure cases (empty text, missing field, non-string input). This gives a reviewer a one-click way to verify both the happy path and the edge cases.

#	Test Case	Expected Outcome
1	Complaint example	200 OK — category: Complaint
2	Query example	200 OK — category: Query
3	Feedback example	200 OK — category: Feedback
4	Other example	200 OK — category: Other
5	Empty text	400 — validation error
6	Missing field	400 — validation error
7	Non-string input	400 — validation error

07

Key Engineering Decisions

Reliability First

- Fallback classifier ensures 100% response availability
- Explicit, typed validation before any AI call is made
- Environment variables kept out of source control

Maintainability

- Strict separation of routes, controllers, and services
- Consistent, predictable JSON error contracts
- Documented setup, usage, and testing in README

A Note on Why This Project Mattered to Me

Beyond meeting the assignment's requirements, I approached this build the way I would approach a real production feature — thinking through failure modes, keeping the code readable for the next engineer, and making sure the system degrades gracefully instead of breaking. I genuinely enjoyed the process of combining a modern AI model with disciplined backend engineering, and it reflects the kind of contributor I aim to be.

I don't just want to build software that works — I want to build software your team can trust, and I would be genuinely excited to bring that same care and curiosity to INT Global Technologies every single day.